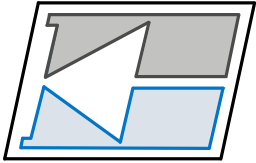


INSTITUTO FEDERAL
Santa Catarina



Departamento Acadêmico de Eletrônica – **DAELN**
IFSC Câmpus Florianópolis

Programação C++

Structs e Unions

Prof. Matheus Leitzke Pinto
matheus.pinto@ifsc.edu.br

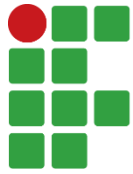
Tipos de dados compostos

- Excluindo classes, os tipos de dados vistos até o momento (int, float, char,...) são denominados tipos de dados primitivos.
- Porém, em muitos casos é necessário a criação de variáveis que representem tipos de dados mais complexos.
- No C++ é possível criar tipos de dados compostos por tipos de dados primitivos, ou mesmo por outros tipos compostos

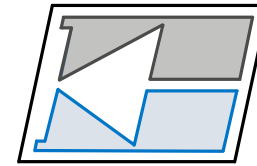
Sumário de aula

- Structs
- Alinhamento de memória
- Unions
- Bit fields





INSTITUTO FEDERAL
Santa Catarina



DAELN

Structs e Unions

Structs

Structs

- Structs são semelhantes a Classes, contudo podem ser compostas apenas por atributos.
- Além disso, podem ser constituídas de métodos e não necessitam de um construtor.
- Outras diferenças:
 - Enquanto os membros de uma classe são privadas por padrão, os membros de uma struct são públicas por padrão
 - Não é possível aplicar templates em structs (não será visto nessa disciplina.)

Structs

- Exemplo

```
#include <iostream>
#include <string>
#include <cstring>

using namespace std;

#define MAX_TAM_NOME 100
#define MAX_TAM_CURSO 100

struct Aluno
{
    unsigned idade;
    unsigned matricula;
    char nome[MAX_TAM_NOME];
    char curso[MAX_TAM_CURSO];
};

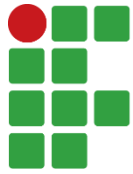
int main()
{
    Aluno eu;

    eu.idade = 23;
    eu.matricula = 1396778;
    strcpy( eu.nome, "Matheus Leitzke Pinto" );
    strcpy( eu.curso, "Engenharia de Computação");

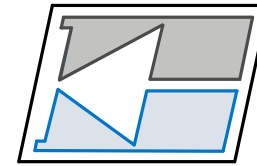
    cout << "nome: " << eu.nome << endl;
    cout << "idade: " << eu.idade << endl;
    cout << "curso: " << eu.curso << endl;
    cout << "matrícula: " << eu.matricula << endl;
}
```

Structs

- Em geral, structs são mais simples e mais fáceis de entender do que classes, especialmente para estruturas de dados simples que não requerem encapsulamento ou métodos complexos.
- Além disso, structs em C++ que possuam apenas atributos são compatíveis com structs em C, o que pode ser útil ao trabalhar com código legado em C ou ao precisar interagir com bibliotecas escritas em C.



INSTITUTO FEDERAL
Santa Catarina



DAELN

Structs e Unions

Alinhamento de memória

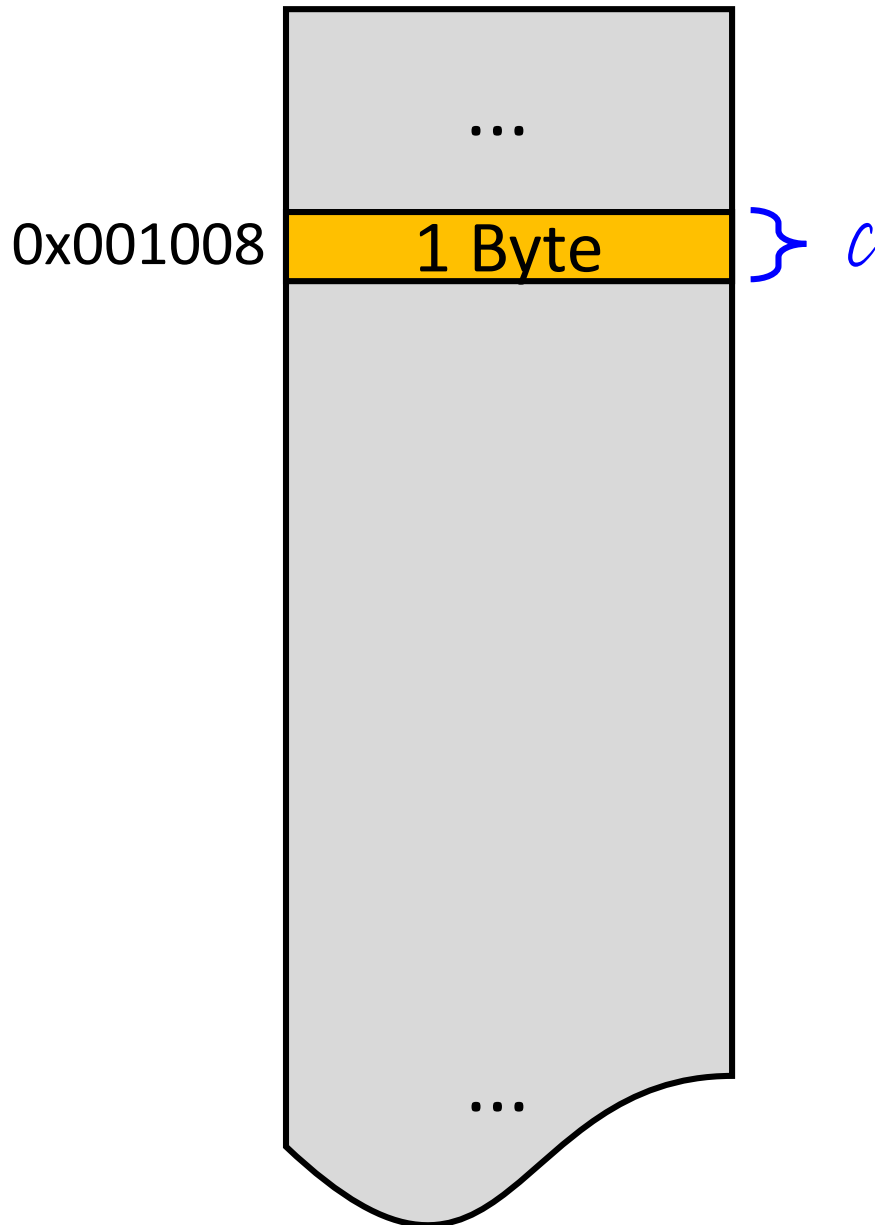
Alinhamento de memória

- O **alinhamento de memória** é uma prática usada pelo **compilador** para otimizar o acesso à memória e garantir a compatibilidade com a arquitetura da CPU.
- Consiste em deixar as variáveis em endereços que sejam múltiplos do seu tamanho.
- Exemplo:
 - Um int (4 Bytes) vai ocupar endereços múltiplos de 4 Bytes – 0x001000, 0x001004, 0x001008, etc.
 - Um char (1 Byte) vai ocupar qualquer endereço, pois qualquer endereço é múltiplo de 1 Byte.

Alinhamento de memória

- O alinhamento é invisível ao programador quando declaramos variáveis primitivas isoladas.
- Contudo, podemos ver os efeitos que esse alinhamento causa no tamanho de structs e classes.
- Também, podemos ter um certo controle de como o alinhamento será feito para otimizar o tamanho em memória.

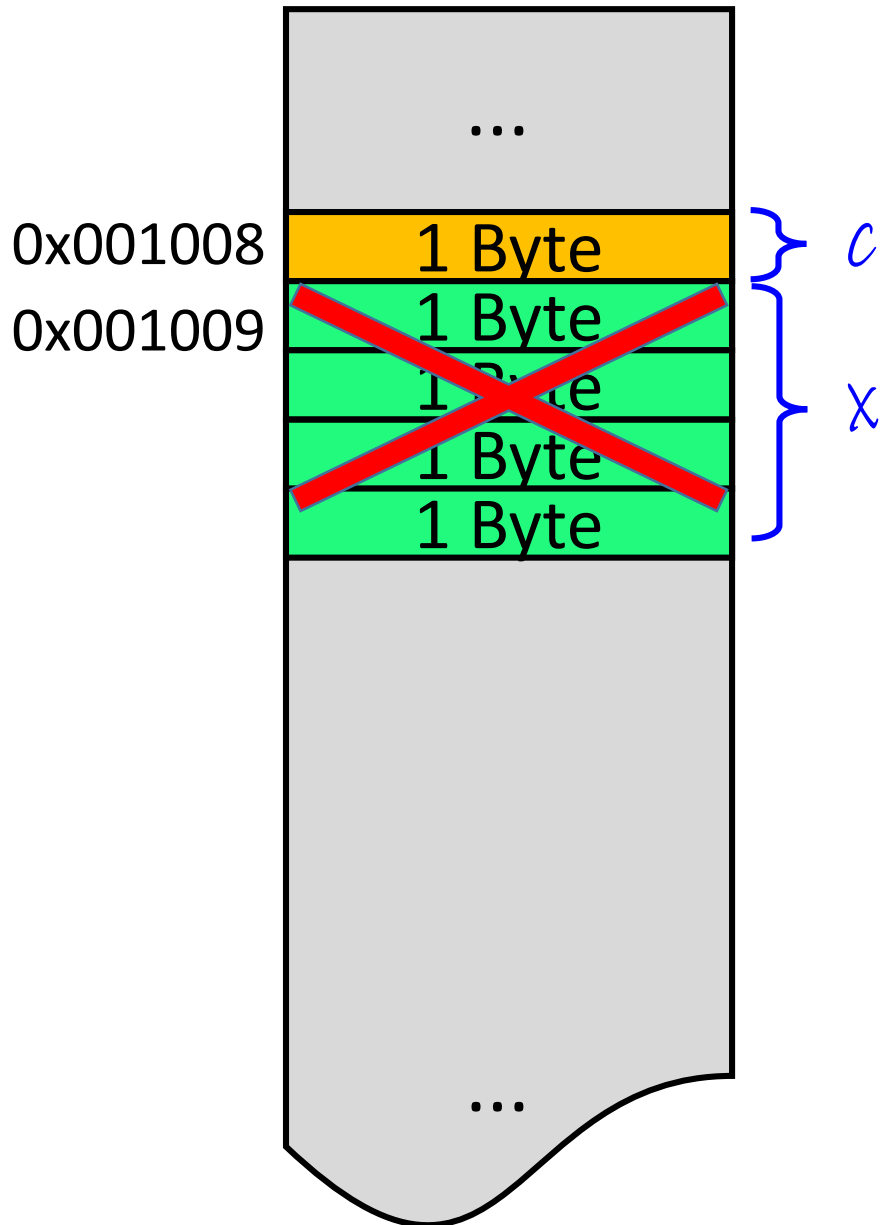
Alinhamento de memória



```
struct S  
{  
    char c;  
};
```

O campo "c" vai ocupar qualquer posição de memória disponível.

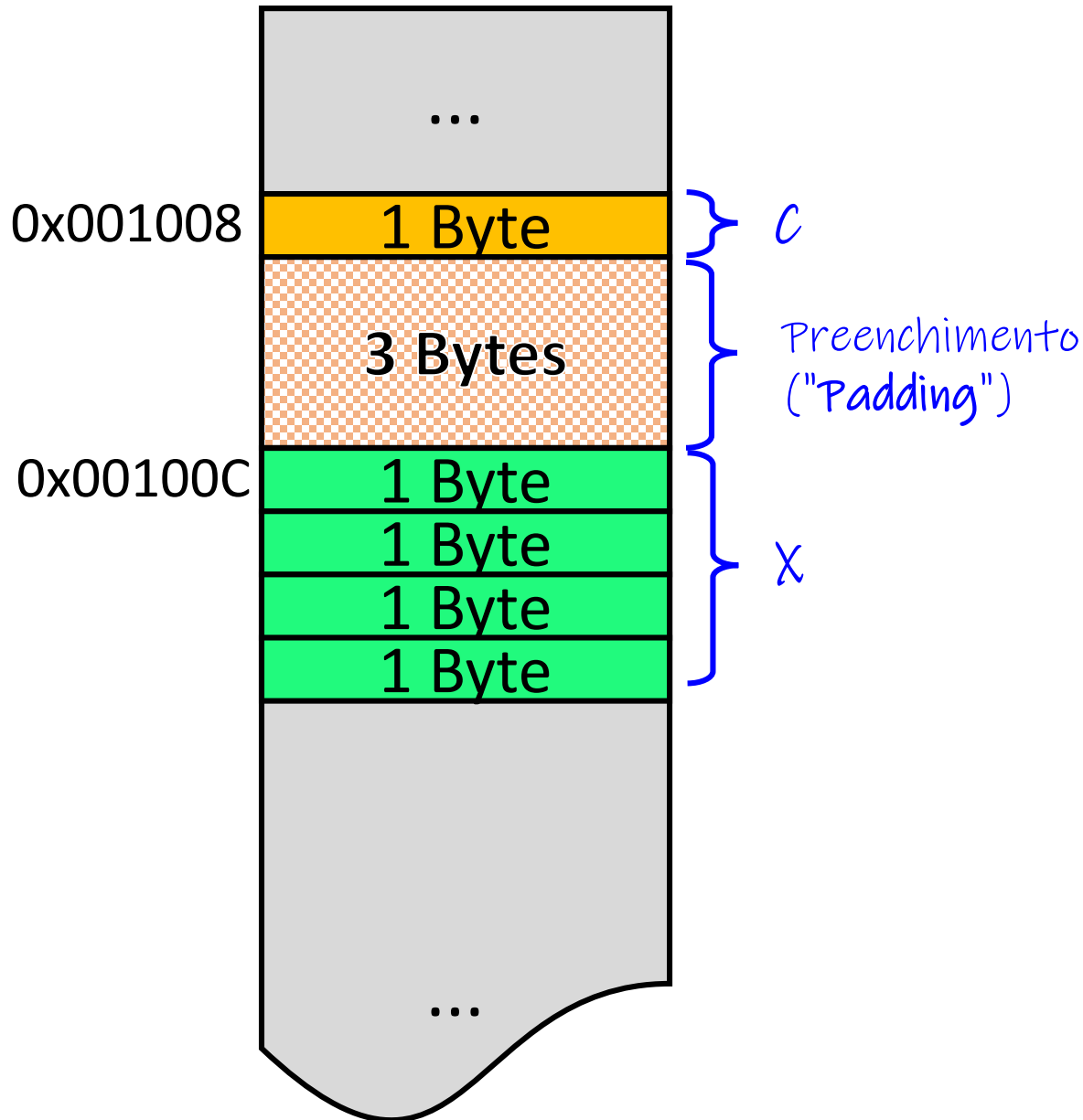
Alinhamento de memória



```
struct S
{
    char c;
    int x;
};
```

O inteiro não vai ocupar a posição 0x001009, pois não é múltiplo de 4 Bytes.

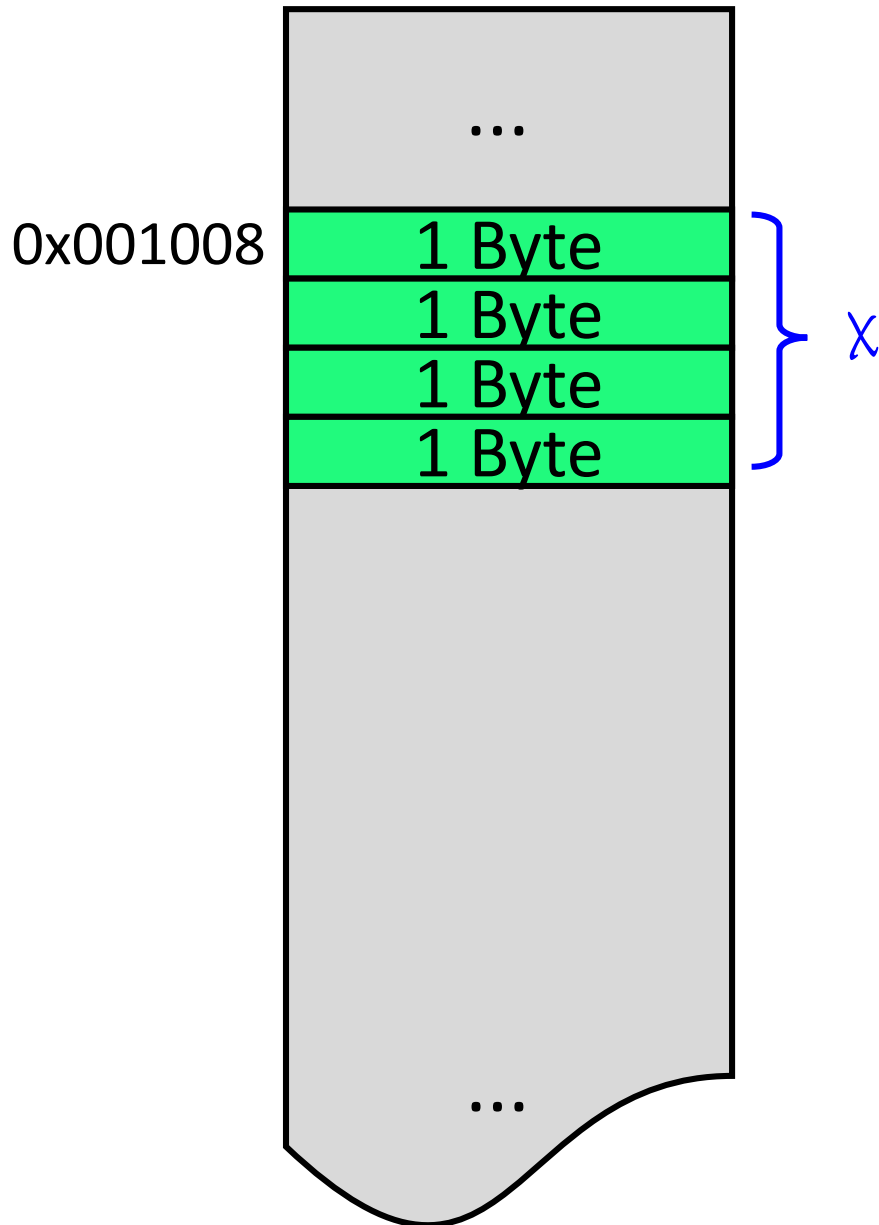
Alinhamento de memória



```
struct S
{
    char c;
    int x;
};
```

Agora sim, "x" vai ficar em uma posição correta (*alinhado*), mas o compilador deve garantir que o endereço de "c" seja múltiplo de 4.

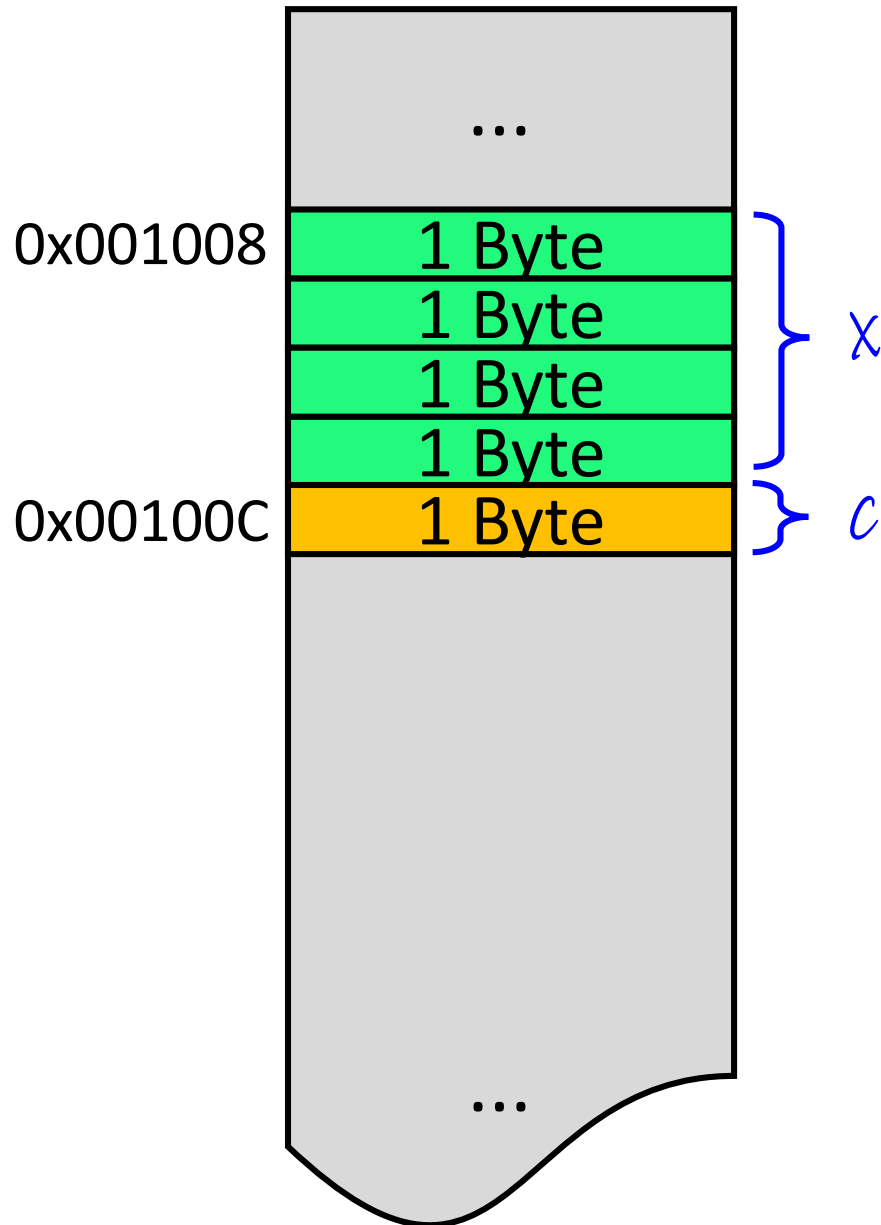
Alinhamento de memória



```
struct S  
{  
    int x;  
};
```

O compilador vai, de alguma forma, **alinhar** o inteiro em um endereço múltiplo 4

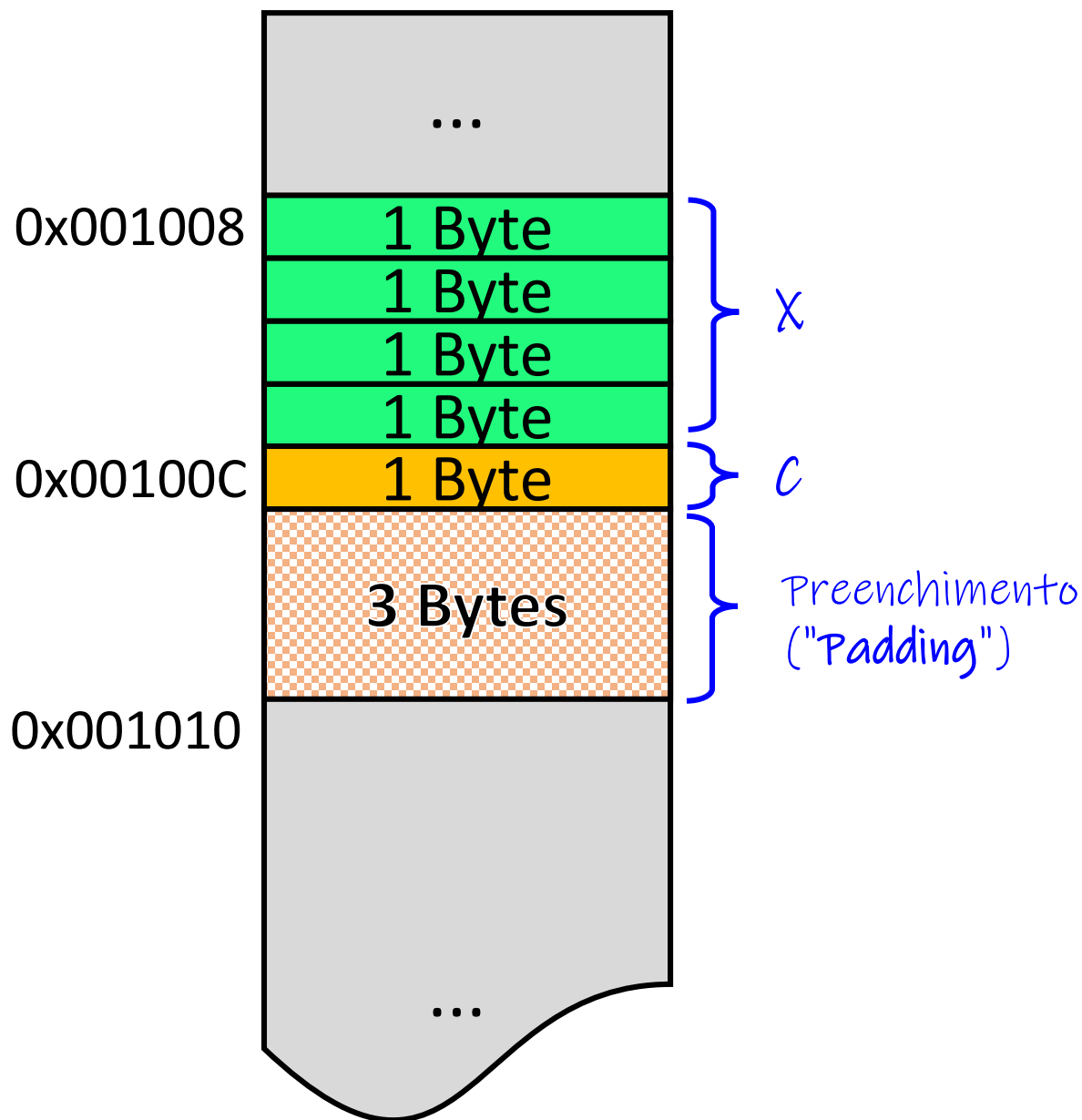
Alinhamento de memória



```
struct S
{
    int x;
    char c;
};
```

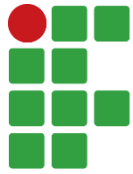
O campo "c" vai ocupar a próxima posição do byte, não importando onde for ficar.

Alinhamento de memória

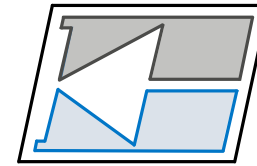


```
struct S
{
    int x;
    char c;
};
```

Contudo, o "padding" é ainda feito para garantir que se um vetor dessa struct for criado, cada uma vai iniciar em um múltiplo de 4.



INSTITUTO FEDERAL
Santa Catarina



DAELN

Structs e Unions

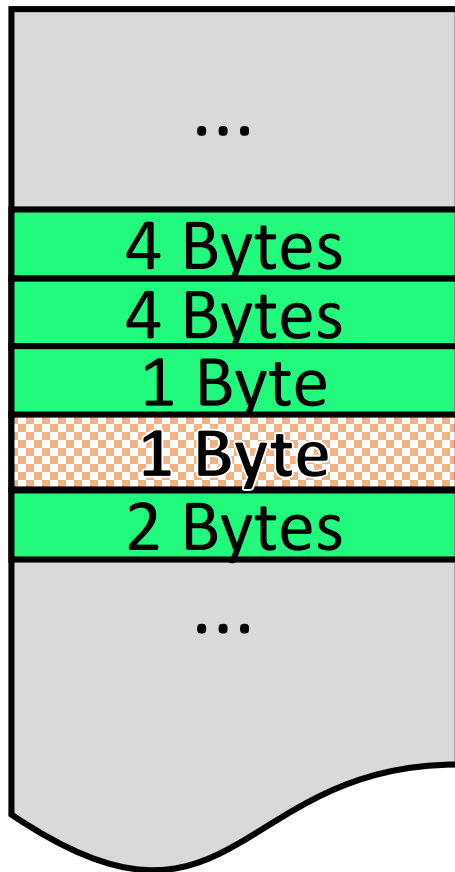
Unions

Unions

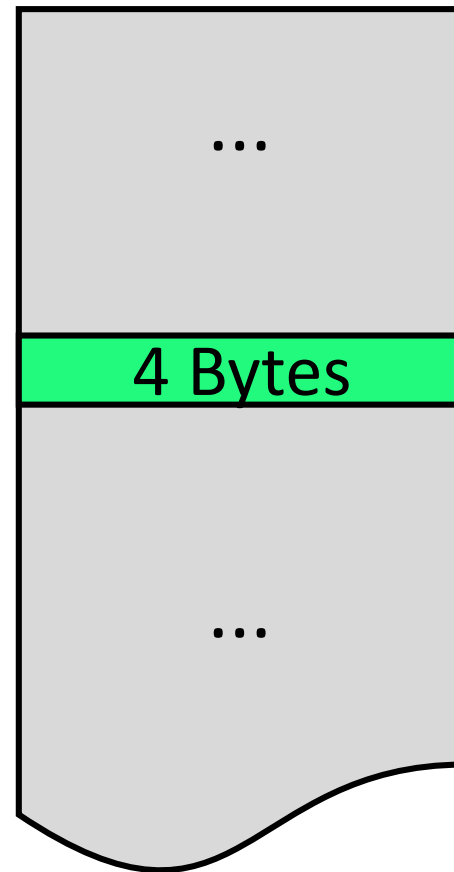
- **Unions** são semelhantes às Structs, contudo utilizam um mesmo bloco de memória para acomodar todos os seus atributos.
- Ou seja, todos os atributos são a mesma coisa dentro da memória.
- O bloco de memória alocado para uma Union é igual ao tamanho do maior atributo.

Unions

- Mapa de memória – Struct vs Union



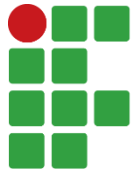
```
struct S
{
    int x;
    unsigned y;
    char c;
    short s;
};
```



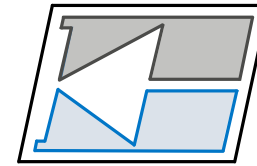
```
union S
{
    int x;
    unsigned y;
    char c;
    short s;
};
```

Unions

- Unions são interessantes quando queremos economizar área de memória.
- Nesse caso, Unions são úteis em aplicações que usam diversas variáveis, mas apenas uma por vez dentro do programa, onde os valores podem ser sobrescritos.
- De fato, existem diversas aplicações para Unions que vão muito além do escopo da disciplina.



INSTITUTO FEDERAL
Santa Catarina



DAELN

Structs e Unions

Bit fields

Bit fields

- Um **Bit field** é um atributo de uma classe ou struct com tipo primitivo (int, char, float, etc,) porém com um tamanho de bits específico.
- Por exemplo, podemos fazer com que um atributo seja do tipo int, mas que ocupe 3 bytes na memória e não 4 bytes.

Bit fields

- Exemplo:

```
#include <iostream>
#include <string>
#include <cstring>

using namespace std;

#define MAX_TAM_NOME 100
#define MAX_TAM_CURSO 100

struct Aluno
{
    unsigned idade : 24; // bit field de 3 bytes
    unsigned matricula : 24; // bit field de 3 bytes
    char nome[MAX_TAM_NOME];
    char curso[MAX_TAM_CURSO];
};

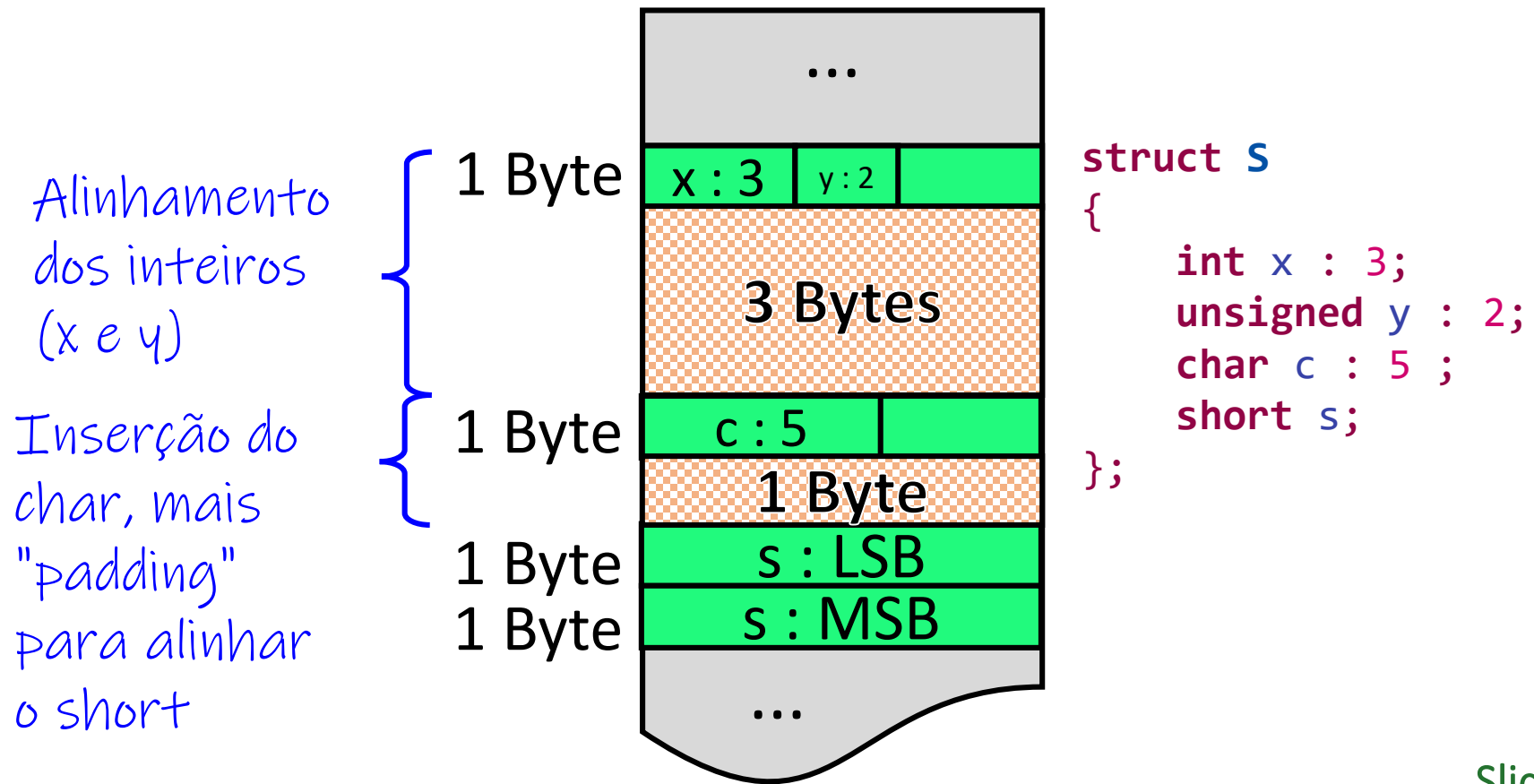
int main()
{
    Aluno eu;

    eu.idade = 23;
    eu.matricula = 1396778;
    strcpy( eu.nome, "Matheus Leitzke Pinto" );
    strcpy( eu.curso, "Engenharia de Computação");

    cout << "nome: " << eu.nome << endl;
    cout << "idade: " << eu.idade << endl;
    cout << "curso: " << eu.curso << endl;
    cout << "matrícula: " << eu.matricula << endl;
}
```

Bit fields

- Como a memória é separada em campos de 1 byte, *bit fields* irão compartilhar uma mesma posição de memória, mas em bits diferentes.

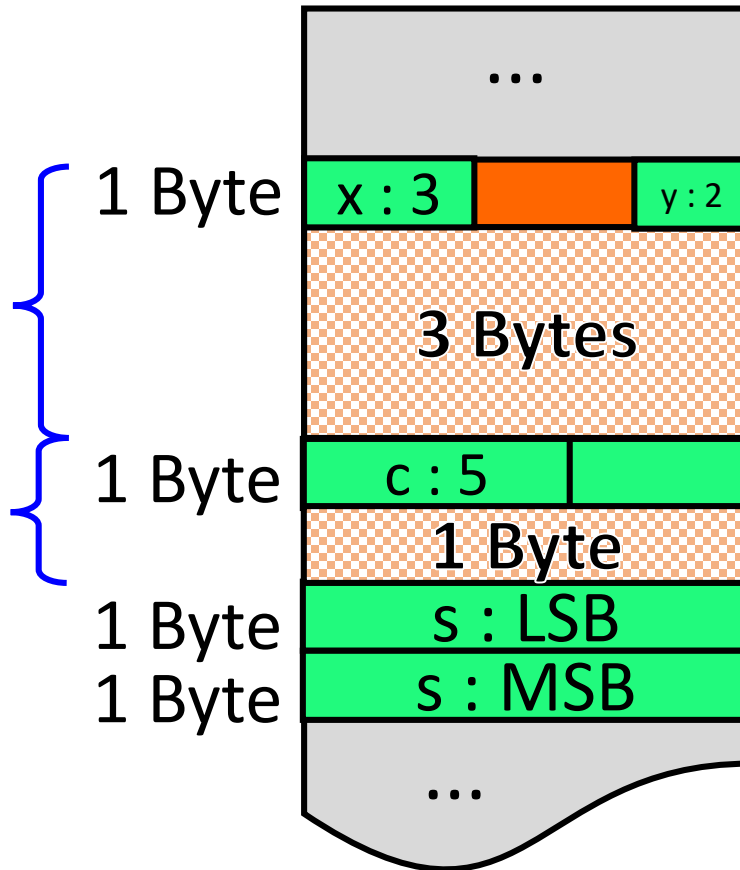


Bit fields

- Como a memória é separada em campos de 1 byte, *bit fields* irão compartilhar uma mesma posição de memória, mas em bits diferentes.

Alinhamento dos inteiros (x e y)

Inserção do
char, mais
"padding"
para alinhar
o short



Indica apenas um espaço vazio de bits, que não serão utilizados.

```
struct S
{
```

```
int x : 3;
int   : 3;
unsigned y : 2;
char c : 5 ;
short s;
```

} ;